

AgentFlow Codex / Claude Code 实现指令文档

用于指导 AI 编程工具分阶段实现 MVP。建议复制到 Codex 或 Claude Code 分步执行。

一、推荐技术栈

- 前端/全栈：Next.js 14+ App Router + TypeScript。
- 样式：Tailwind CSS + shadcn/ui + lucide-react。
- 状态与数据：MVP 使用 mock 数据和 localStorage；二阶段接入 Prisma + PostgreSQL。
- 认证：MVP 可先不做登录，使用角色切换；二阶段接入 NextAuth。
- 部署：Vercel；托管页使用动态路由 /site/[slug]。
- AI 调用：MVP 用模拟日志；二阶段接入 OpenAI/Claude API 或本地 CLI。

二、项目目录建议

```
agentflow-mvp/  
  app/  
    page.tsx  
    services/page.tsx  
    request/page.tsx  
    orders/[id]/page.tsx  
    preview/[id]/page.tsx  
    admin/[id]/page.tsx  
    operator/page.tsx  
    cli/page.tsx  
    site/[slug]/page.tsx  
  components/  
    ServiceCard.tsx  
    Stepper.tsx  
    OrderTimeline.tsx  
    PhonePreview.tsx  
    TaskSpecPanel.tsx  
    OperatorChecklist.tsx  
    TerminalPanel.tsx  
  lib/  
    mock-data.ts  
    types.ts  
    task-spec.ts  
    validation-rules.ts  
  styles/  
    globals.css
```

三、核心数据模型

```
type ServicePackage = {  
  id: string;  
  name: string;  
  price: number;
```

```
    included: string[];
    excluded: string[];
    maintenance: string;
  };

type Order = {
  id: string;
  packageId: string;
  status: 'submitted' | 'generating' | 'checking' | 'operator_review' | 'published' | 'confirmed';
  merchant: MerchantProfile;
  taskSpec: TaskSpec;
  previewUrl?: string;
  operatorId?: string;
};

type TaskSpec = {
  taskType: 'merchant_landing_page' | 'event_form_page' | 'product_page';
  deliverables: string[];
  acceptance: string[];
  excluded: string[];
};
```

四、给 Codex / Claude Code 的总指令

你可以把下面这段复制给 Codex 或 Claude Code 作为项目总任务：

请你实现一个 Next.js + TypeScript + Tailwind CSS 的 AgentFlow MVP。

产品定位：AI 驱动的托管式数字交付网络。

MVP 场景：小微商家手机端宣传页托管服务。

要求实现页面：首页、服务包选择、需求填写、订单进度、成品预览与管理后台、Operator 工作台、CLI/Skill 演示页、托管落地页。

要求使用 mock 数据，不接真实支付，不接真实 AI API。

UI 风格：现代 SaaS，蓝白色，卡片布局，移动端优先，强调“服务包 + 托管交付 + Operator 兜底”。

交互要求：用户可以选择服务包、填写门店信息、提交后生成订单；订单页展示交付进度；预览页展示手机端营销页；后台页允许编辑电话、营业时间、地址；Operator 页展示任务队列、执行计划和质检清单；CLI 页展示模拟命令和日志。

请先生成完整项目结构，再逐步实现组件和页面，最后检查 TypeScript、lint 和响应式布局。

五、分阶段执行指令

阶段 1：初始化项目

创建 Next.js 14 App Router 项目，安装 Tailwind、shadcn/ui、lucide-react，设置基础布局、字体和主题色。

阶段 2：定义数据结构

在 lib/types.ts 定义 ServicePackage、Order、TaskSpec、MerchantProfile、OperatorTask 等类型，并在 mock-data.ts 写入示例数据。

阶段 3：首页和服务包页

实现首页 hero、双入口、服务包卡片、交付流程图和套餐边界说明。

阶段 4：需求填写页

实现四步表单：基本信息、页面内容、风格素材、确认交付；右侧实时显示 TaskSpec。

阶段 5：订单进度页

实现时间线：需求提交、Agent 生成、自动质检、Operator 兜底、发布托管、用户确认。

阶段 6：成品预览与后台

实现手机端营销页预览、托管链接、可编辑字段后台和保存重新发布按钮。

阶段 7：Operator 工作台

实现任务队列、Agent 执行计划、质检清单、终端日志、确认提交/退回按钮。

阶段 8：CLI/Skill 演示页

实现 TerminalPanel，展示 agentflow login/recommend/claim/plan/run/check/submit 全流程。

阶段 9：质量检查

运行 npm run lint 和 npm run build，修复类型错误、响应式问题和可访问性问题。

六、关键页面验收标准

页面	验收标准
首页	能解释产品定位，双入口清晰，突出托管式交付而不是接单市场
服务包页	每个套餐有价格、包含内容、不包含内容、维护边界
需求填写页	表单字段完整，右侧 TaskSpec 实时变化
订单进度页	用户能理解当前处于哪一步、谁在处理、下一步是什么
预览后台页	能看到托管页面、可编辑字段和保存重新发布流程
Operator 页	Operator 能看到任务规格、计划、质检项和提交动作
CLI 页	命令日志完整体现后台生产工具逻辑